

Telegram Clone

张天睿 · 智能科学与技术 · 2027届 · 求职意向：游戏运维开发

仿 Telegram 实时通讯平台 — FastAPI + WebSocket + AI Bot

在线演示 (Swagger UI)

GitHub 源码

项目概述

一款功能完整的仿 Telegram 实时通讯后端系统。基于 FastAPI 全异步架构，支持 WebSocket 实时私聊/群聊消息推送，内置 DeepSeek AI Bot，支持多轮连续对话、Tool Calling（函数调用）和 RAG 文档问答。项目已部署上线，运行在阿里云 ECS（Ubuntu + Nginx + systemd），线上可直接体验。

角色

全栈后端开发（独立完成）

代码规模

约 4000 行 Python（含测试）

项目周期

约 2 个月（迭代开发）

线上地址

mytelegram.xyz/docs

技术栈

FastAPI Web 框架	SQLAlchemy (Async) ORM + asyncpg	PostgreSQL 生产数据库
Redis 缓存 + 在线状态	WebSocket 实时消息推送	JWT + bcrypt 认证 & 安全
DeepSeek API AI 对话引擎	ChromaDB 向量数据库	sentence-transformers 文本向量化
LangChain/LangGraph Agent 编排	Nginx 反向代理	Docker 容器化部署
systemd 进程守护	pytest 自动化测试	

系统架构

分层架构：路由层 → 业务层 → 数据层，职责清晰，可测试性强。



核心功能

用户认证

手机号注册/登录，JWT Token 鉴权，bcrypt 密码哈希，Bearer Token 中间件拦截验证

好友系统

好友请求发送→接受/拒绝→双向关系建立，支持搜索用户、删除好友

私聊消息

WebSocket 实时收发，支持文本消息、已读标记、消息撤回（2分钟内）、消息搜索

群聊系统

创建群组、成员管理（添加/移除/角色权限）、群消息多播广播、群消息历史分页

AI Bot

内置 AI Bot 用户，发消息即可对话。支持 20 条上下文记忆，可调用工具查天气/时间/搜索

RAG 文档问答

上传 TXT/PDF → 分块 → sentence-transformers 向量化 → ChromaDB → 检索增强生成回答

在线状态

Redis TTL 300s 管理在线状态，WebSocket 心跳 120s 续期，支持查询任意用户在线状态

离线消息

接收方不在线时消息存入 Redis 列表，上线后通过 HTTP API 拉取并清空

API 接口覆盖

模块	接口数	说明
认证 (auth)	2	注册、登录
用户 (users)	4	个人信息 CRUD、搜索用户
好友 (friends)	5	请求发送/接受/拒绝、列表、删除
消息 (messages)	5	发送、历史分页、已读、撤回、搜索
会话 (conversations)	1	私聊+群聊会话列表聚合
群组 (groups)	8	CRUD + 成员管理 + 权限控制
文件 (upload)	1	文件上传（类型/大小校验）
AI (ai)	4	文档上传/列表/删除、RAG 问答
实时 (ws)	1	WebSocket 全双工通信
其他	3	离线消息拉取、在线状态查询、健康检查
合计	34	REST + WebSocket

AI 技术亮点：Tool Calling 流程

AI Bot 不仅能聊天，还能根据用户意图自动调用外部工具，获取实时数据后再生成回答。

用户: "北京今天天气怎么样?"

- AI 分析意图 → 判断需要调用 `get_weather` 工具
- 自动调用 `get_weather(city="北京")` → 获取天气数据
- 将天气数据拼入上下文 → AI 生成自然语言回答
- "北京今天晴，气温 22-30°C，适合出行。"

已实现工具: `get_time`(世界时间) / `get_weather`(天气) / `web_search`(Tavily联网) / `search_knowledge`(文档检索)

AI 技术亮点：LangChain/LangGraph Agent 编排

基于 LangGraph 实现可配置的 AI Agent，支持工具调用链编排，可通过配置切换手写版 Agent 与 LangChain 版 Agent。

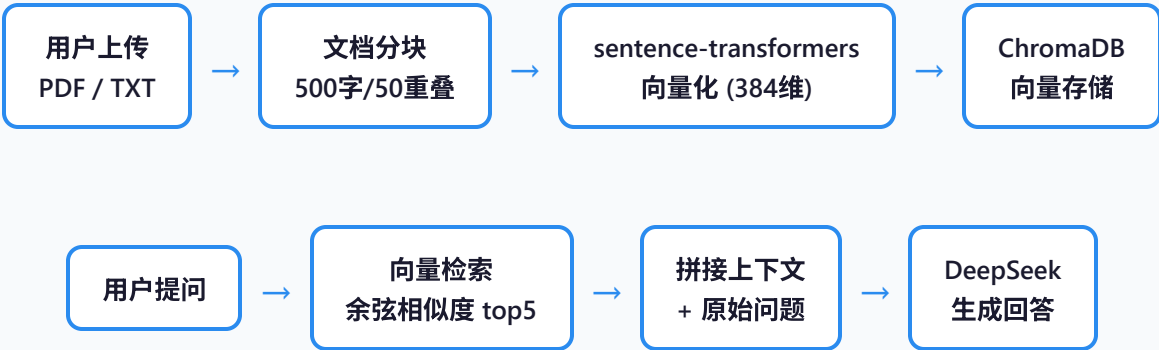
Agent 架构

- LangGraph StateGraph 管理对话状态流转
- 4 个 Tool 节点: `get_time` / `get_weather` / `web_search` / `search_knowledge`
- DeepSeekJsonWrapper 剥离 JSON 围栏，适配 DeepSeek API
- 条件边路由: 根据 LLM 决策调度工具调用或生成最终回复

配置切换

- 环境变量 `AI_BACKEND` 控制: `native` / `langchain`
- 手写版: 轻量级 Function Calling，无框架依赖
- LangChain 版: 完整的 Agent 编排，可扩展工具和记忆
- 两套实现共享相同的 Tool 定义和 LLM 调用接口

AI 技术亮点：RAG 文档问答管道



AI 技术亮点：RAGAS 评测体系

使用 RAGAS 框架对 RAG 文档问答进行系统性质量评测，四项指标零报错，验证了检索和生成流程的可靠性。

指标	分数	说明
faithfulness（忠实度）	0.73	回答是否忠于检索到的上下文，有无编造
answer_relevancy（答案相关性）	0.77	回答与问题的相关程度
context_recall（上下文召回率）	0.79	检索到的上下文覆盖参考答案的程度
context_precision（上下文精确度）	0.36	检索结果中相关文档的排名质量

* 评测过程中解决了 HuggingFace 直连超时（自研 ceshi_Wrapper 走 hf-mirror.com）和 DeepSeek JSON 围栏解析等技术难点。

技术亮点：WebSocket 实时通信

基于 FastAPI 原生 WebSocket 实现的实时消息系统，一个用户支持多设备同时在线。

连接管理

- defaultdict[user_id, list[WebSocket]] 支持多设备
- 连接时 JWT 鉴权，拒绝无效 Token
- 断开时自动清理 + Redis 下线

消息路由

- 私聊: 查在线状态 → 在线直接推送 / 离线存 Redis
- 群聊: 查在线集合 ∩ 群成员 → 多播所有在线成员
- AI Bot: receiver 为 ai_bot 时触发 DeepSeek 流式回复

```
WebSocket 消息类型:
message      → 发送私聊消息      new_message → 接收新消息
group_message → 发送群消息      group_broadcast → 群消息广播
typing       → 输入状态          online_status → 在线状态变更
ping/pong    → 心跳保活 (120s)  read          → 已读回执
```

技术亮点：WebSocket 性能压测

使用 Python websockets 库编写压测脚本，模拟真实用户并发连接与消息收发，验证系统在高并发下的稳定性。

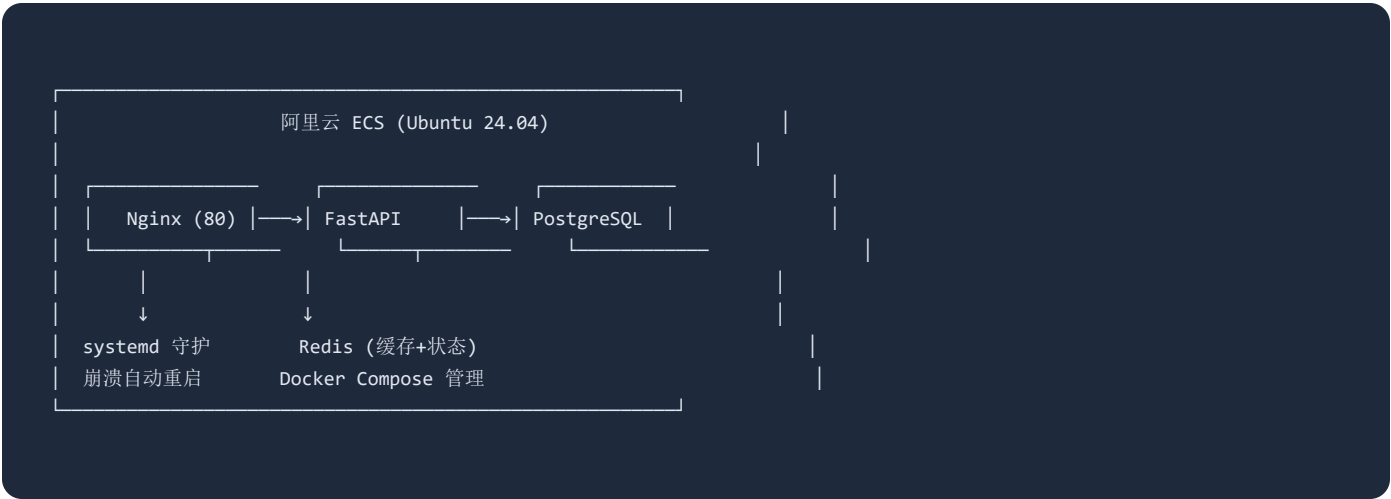
指标	结果
并发连接数	1000
成功率	100%
连接速率	183 conn/s
测试工具	自研 Python 压测脚本 (tests/ws_bench.py)

数据库设计

表名	核心字段	关系
users	id(UUID), phone, username, display_name, hashed_password, last_seen	-
friends	user_id, friend_id, status(pending/accepted/rejected)	FK → users
messages	id(UUID), sender_id, receiver_id, content, msg_type, is_read, is_revoked, created_at	FK → users
groups	id(UUID), name, description, owner_id, created_at	FK → users
group_members	group_id, user_id, role(owner/admin/member), joined_at	FK → groups, users
group_messages	id(UUID), group_id, sender_id, content, msg_type, is_revoked, created_at	FK → groups, users
documents	id(UUID), user_id, filename, file_type, chunk_count, uploaded_at	FK → users

* 所有 ID 使用 UUID 类型，时间字段使用 UTC，密码使用 bcrypt 哈希存储

部署架构



部署细节：容器化 + 进程管理

Docker Compose 服务拓扑

- **app** — FastAPI + uvicorn，映射 8000 端口
- **db** — PostgreSQL 16，持久化 volume
- **redis** — Redis 7，缓存 + WebSocket 在线状态
- 服务间通过 Docker network 内部通信

Nginx 反向代理

- 静态文件直接 serve (uploads/、portfolio.html)
- /api/* 和 /ws/* 反向代理到 uvicorn
- WebSocket 升级：proxy_set_header Upgrade \$http_upgrade
- SSL 证书 via Let's Encrypt / Certbot

systemd 进程守护

- telegram-clone.service 管理 uvicorn 进程
- Restart=always 崩溃自动重启
- RestartSec=5 重启间隔 5 秒
- 日志集成 journald，可通过 journalctl -u telegram-clone 查看

测试策略

单元测试 / 集成测试

- 独立 SQLite test.db，与开发环境隔离
- httpx.ASGITransport 不启动真实服务器
- 依赖注入覆盖 (dependency_overrides)
- 每个测试前后自动建表/删表

覆盖模块

- 用户认证：注册/登录/JWT验证
- 好友系统：请求流程完整链路
- 消息系统：发送/历史/撤回
- RAG 评估：RAGAS 框架评估检索质量
- WebSocket 压测：websockets 并发性能

技术决策 & 思考

决策点	选择	理由
Web 框架	FastAPI (而非 Django/Flask)	原生 async/await、自动 OpenAPI 文档、WebSocket 一等公民
数据库驱动	asyncpg (而非 psycopg2)	全异步链路避免阻塞事件循环，配合 SQLAlchemy Async
密码哈希	bcrypt (而非 passlib)	passlib 已停止维护，直接使用 bcrypt 库更简单可靠
向量数据库	ChromaDB (而非 Pinecone/Milvus)	轻量、无需额外服务、适合小规模 RAG 场景
数据库迁移	手动 ALTER TABLE (而非 Alembic)	项目规模小，手动变更足够灵活，避免迁移文件管理开销
进程管理	systemd (而非 Supervisor/Docker)	Linux 原生、开机自启+崩溃重启、日志集成 journald

个人收获与成长

异步编程深入理解

从同步思维切换到 async/await 范式，理解了事件循环、协程调度、连接池管理，以及为什么全链路异步才有意义

WebSocket 协议实战

掌握了 WebSocket 握手机制、心跳保活、断线重连、多设备连接管理、群消息广播路由等生产级实时通信要点

AI 应用开发全流程

从 API 调用到 Function Calling，再到 RAG 检索增强生成，完整理解了 LLM 应用从 prompt 到 production 的链路

运维部署能力

云服务器初始化、Nginx 反向代理配置、systemd 进程管理、HTTPS 证书，从本地开发到线上可用的完整交付

张天睿 — 智能科学与技术 · 2027届

[在线演示](#) · [GitHub](#)

求职意向：游戏运维开发 · Built with FastAPI + PostgreSQL + Redis + WebSocket + DeepSeek AI